

Introduction

In a multithreaded environment, ensuring that shared resources are accessed safely is critical. But not every synchronization mechanism is designed to handle real-world constraints effectively. When it comes to designing scalable and responsive systems, certain limitations begin to surface.

Problem Statement

Let's consider a real-world scenario from **BookMyShow**. Suppose a user begins booking a movie ticket, and this booking operation runs inside a thread. However, midway through the booking process, the user goes idle — maybe they switch tabs or walk away from the screen.

Now, if this operation is guarded using the **synchronized** keyword in Java, the thread will continue holding the lock for the entire duration. Other users trying to access the same seat will be blocked **indefinitely** until that thread completes. This becomes a serious bottleneck in systems that expect real-time performance and responsiveness.

Why not just rely on **synchronized**?

- **No timeout support:** the lock waits forever.
- No explicit control over acquiring/releasing the lock.
- You can't interrupt a thread stuck waiting for a lock.
- **No guarantee of fairness:** Some threads may wait longer than others.

Such limitations make **synchronized** unsuitable for modern concurrent applications where responsiveness and control are essential.

To solve this issue, a better alternative like a **Mutex** can be used. But before we dive into it, we first need to understand the difference between a **Lock** and a **Mutex**.

Lock Vs Mutex

Lock	Mutex
A general term used to achieve mutual exclusion in concurrent programming.	A specific type of lock that enforces ownership semantics .
Lock enforcement is not always strict, i.e., any thread may release the lock.	Only the thread that acquires the mutex can release it.
Often implemented using the synchronized keyword in Java.	Implemented using constructs like ReentrantLock in Java, which behaves like a mutex.
In programming, one thread might unlock what another thread locked.	Only the thread that locked it is allowed to unlock it, ensuring ownership consistency.

Real-life analogy: Public washroom – anyone can lock or unlock.	Real-life analogy: Your home – only you (owner) have the key to lock/unlock.
--	---

ReentrantLock – A Better Alternative to **synchronized**

We have already seen how the **synchronized** keyword can leave threads waiting forever when a user simply goes idle in the middle of a critical section.

This is where **ReentrantLock** steps in to give us finer, explicit control over locking behaviour.

What is ReentrantLock?

ReentrantLock (in `java.util.concurrent.locks`) is a mutual-exclusion lock with **ownership semantics**: the thread that acquires it is the only one allowed to release it. The same thread may acquire the lock multiple times (“re-enter”) without deadlocking – hence reentrant.

When Should You Use It?

Use Case	Why ReentrantLock Helps
Explicit control over locking and unlocking	You can decide exactly when to acquire and release the lock, unlike synchronized which is block-scoped.

Attempt to acquire a lock with or without timeout	Methods like <code>tryLock()</code> and <code>tryLock(long, TimeUnit)</code> prevent threads from being blocked indefinitely.
Fine-grained synchronization	Enables locking only the necessary portions of code, reducing contention and increasing concurrency.

Code Walk-through

Below is a simplified ticket-booking example that replaces `synchronized` with `ReentrantLock`.

```
import java.util.concurrent.locks.ReentrantLock;

class TicketBooking {

    // Number of seats initially available

    private int availableSeats = 1;

    // Dedicated lock for this shared resource

    private final ReentrantLock lock = new ReentrantLock();

    // Public method to book a ticket

    public void bookTicket(String user) {

        System.out.println(user + " is trying to book...");
```

```
// Acquire the lock – blocks until available

lock.lock();

try {

    System.out.println(user + " acquired lock.");

    // Critical section: check and update shared state

    if (availableSeats > 0) {

        System.out.println(user + " successfully booked the ticket.");

        availableSeats--;

    } else {

        System.out.println(user + " could not book the ticket. No seats
left.");

    }

} finally {

    // Always release the lock in a finally block

    System.out.println(user + " is releasing the lock.");

    lock.unlock();

}

}
```

```
class Main {  
  
    public static void main(String[] args) {  
  
        // Shared instance of TicketBooking  
  
        TicketBooking bookingSystem = new TicketBooking();  
  
  
        // Create two threads representing two users trying to book at the  
        same time  
  
        Thread user1 = new Thread(() -> bookingSystem.bookTicket("User  
1"));  
  
        Thread user2 = new Thread(() -> bookingSystem.bookTicket("User  
2"));  
  
  
        // Start both threads  
  
        user1.start();  
  
        user2.start();  
  
  
        // Wait for both threads to finish  
  
        try {  
  
            user1.join();  
  
            user2.join();  
  
        } catch (InterruptedException e) {  
  
            System.out.println("Thread interrupted: " + e.getMessage());  
  
        }  
    }  
}
```

```
}  
  
}  
  
}
```

Key Points:

- `lock.lock()` – blocks the calling thread until it secures exclusive access.
- `lock.unlock()` – must always execute (hence the `finally` block) to avoid deadlocks.

Even though this removes the indefinite blocking of *other* threads once the critical section ends, it **still does not handle** the case where the user acquires the lock and then sits idle for a long time *inside* the critical section. To overcome that, we need a non-blocking approach as discussed in the next section.

Expiring Reentrant Lock – Auto-Releasing Idle Threads

Previously, a user who went idle while holding the lock kept everyone else waiting.

The code below adds an expiry timer so the lock is freed automatically when such scenarios arise.

```
import java.util.concurrent.*;
```

```
import java.util.concurrent.locks.ReentrantLock;
```

```
// _____ ExpiringReentrantLock
```

```
// Lock with a built-in "auto-release after N ms" timer
```

```
class ExpiringReentrantLock {
```

```
    // underlying mutual-exclusion lock
```

```
    private final ReentrantLock lock = new ReentrantLock();
```

```
    // single-thread scheduler to run the expiry task
```

```
    private final ScheduledExecutorService scheduler =
```

```
        Executors.newSingleThreadScheduledExecutor();
```

```
    // flag that tells the expiry task a timed lock is still active
```

```
    private volatile boolean isLocked = false;
```

```
    // Tries to acquire immediately; if successful, schedules auto-unlock
```

```
    public boolean tryLockWithExpiry(long timeoutMillis) {
```

```
        // attempt immediate acquisition
```

```
        boolean acquired = lock.tryLock();
```

```
        if (acquired) {
```

```

// mark as held under the timer

isLocked = true;

// schedule unlock after timeout

scheduler.schedule(() -> {

    if (lock.isHeldByCurrentThread() || isLocked) {

        System.out.println("Auto-releasing lock after timeout.");

        unlockSafely(); // delegate to common unlock logic

    }

}, timeoutMillis, TimeUnit.MILLISECONDS);

}

return acquired;

}

```

// Releases the lock either by the owner thread or the timer

```

public void unlockSafely() {

    if (lock.isHeldByCurrentThread() || isLocked) {

        isLocked = false; // reset timer flag

        // only the owner may actually call unlock()

        if (lock.isHeldByCurrentThread()) {

```

```

        lock.unlock();

        System.out.println("Lock released.");
    }
}

```

```

// Graceful shutdown for the scheduler

public void shutdown() {
    scheduler.shutdownNow();
}
}

```

```

// _____ Driver code
_____

```

```

public class Main {

    public static void main(String[] args) {

        // shared expiring lock

        ExpiringReentrantLock expLock = new ExpiringReentrantLock();

        /* Idle user grabs the lock, then "goes missing" for 5 s
        (expiry timer is 3 s) */
    }
}

```

```

Thread idleUser = new Thread(() -> {
    if (expLock.tryLockWithExpiry(3000)) {
        try { Thread.sleep(5000); } // simulate long idle
        catch (InterruptedException ignored) {}
        expLock.unlockSafely(); // in case timer fired
    }
}, "IdleUser");

```

/* Active user starts after 1 s and keeps retrying every 1000 ms
until it finally books the ticket once the timer frees the lock */

```

Thread activeUser = new Thread(() -> {
    try { Thread.sleep(1000); } catch (InterruptedException ignored) {}
    while (true) {
        if (expLock.tryLockWithExpiry(3000)) {
            System.out.println("Active user booked!");
            expLock.unlockSafely();
            break;
        } else {
            System.out.println("Active user still waiting...");
            try { Thread.sleep(1000); } catch (InterruptedException ignored) {}
        }
    }
}

```

```

    }

    }, "ActiveUser");

// start threads

idleUser.start();

activeUser.start();


// wait for both to finish
try {

    idleUser.join();

    activeUser.join();

} catch (InterruptedException ignored) {}


// shut down scheduler

expLock.shutdown();

}

}

```

Key Methods Used

- **lock.tryLock()** – Tries to acquire the lock **immediately**, returns **true** if successful, **false** otherwise. Doesn't block.

- `lock.isHeldByCurrentThread()` – Checks whether the **current thread** holds the lock.
- `lock.unlock()` – Releases the lock. Only the **owning thread** may call this.

Key Takeaways

- **Problem Solved:** Prevents a thread from **holding the lock indefinitely** by introducing a timer that **auto-releases** the lock.
- **Immediate Lock Acquisition:** The method `tryLockWithExpiry()` tries to acquire the lock **immediately** using `lock.tryLock()` without blocking.
- **Auto-Release Mechanism:** If a thread acquires the lock, a **scheduled task** is created that **releases** the lock after a specified timeout, if not released manually.
- **Safe Unlocking:** The `unlockSafely()` method ensures that only the thread that owns the lock can unlock it.
- **Resilient Design:** If the owning thread doesn't call `unlockSafely()` (due to idle or crash), the **scheduler ensures cleanup** to maintain system responsiveness.
- **Thread Retry Logic:** The `ActiveUser` retries acquiring the lock periodically until the idle thread's lock expires and is released.

Using `tryLock(timeout, unit)` – Non-Blocking Wait with an Escape Hatch

In the last example we called `lock.tryLock()` which is a **one-shot** attempt that either succeeds or fails immediately.

But what if we're willing to wait a little while, yet **don't want to block forever**? That is exactly what `tryLock(long timeout, TimeUnit unit)` provides.

Code Demo – Ticket Booking with a 2-second timeout

```
import java.util.concurrent.TimeUnit;

import java.util.concurrent.locks.ReentrantLock;


// Booking system that waits up to 2 s for a lock

class TicketBookingTryLock {

    // shared resource: initial seat count

    private int availableSeats = 1;


    // exclusive lock protecting seat updates

    private final ReentrantLock lock = new ReentrantLock();


    // attempts to book a ticket for the given user

    public void bookTicket(String user) {

        System.out.println(user + " is trying to book...");


        // remember whether we actually got the lock

        boolean lockAcquired = false;

        try {

            // wait up to 2 s before giving up
```

```
lockAcquired = lock.tryLock(2, TimeUnit.SECONDS);

if (lockAcquired) {

    System.out.println(user + " acquired lock.");

    // critical section – safe to inspect/update seats
    if (availableSeats > 0) {

        System.out.println(user + " successfully booked the ticket.");

        availableSeats--;

    } else {

        System.out.println(user + " could not book the ticket. No seats
left.");

    }

    // simulate a long operation that holds the lock

    // this helps demonstrate the timeout behavior for the next user

    Thread.sleep(3000);

} else {

    System.out.println(user + " could not acquire lock. Try again
later.");

}

}
```

```
        catch (InterruptedException e) {  
            // restore interrupt status and log  
            Thread.currentThread().interrupt();  
            e.printStackTrace();  
        }  
        finally {  
            // release only if we were the owner  
            if (lockAcquired) {  
                System.out.println(user + " is releasing the lock.");  
                lock.unlock();  
            }  
        }  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
  
        // one shared booking system  
        TicketBookingTryLock booking = new TicketBookingTryLock();
```

```

// first user attempts booking immediately

Thread user1 = new Thread(() -> booking.bookTicket("User 1"));


// second user arrives 500 ms later and may time-out

Thread user2 = new Thread(() -> {

    try { Thread.sleep(500); } catch (InterruptedException ignored) {}

    booking.bookTicket("User 2");

});

user1.start();

user2.start();

}

}

```

Understanding

- **tryLock(long timeout, TimeUnit unit)**: Waits up to the given time for the lock; returns **true** if obtained, otherwise **false**.
- **TimeUnit.SECONDS**: Convenience enum to express the timeout unit cleanly (**SECONDS**, **MILLISECONDS**, etc.).

Key Takeaways

- **Timed Wait**: **tryLock(2, SECONDS)** blocks only up to the limit; the thread remains responsive.

- **Graceful Fallback:** If the lock isn't obtained within the window, the thread can log, retry, or take another path instead of hanging.
 - **Safe Release:** Always check `lockAcquired` before unlocking to avoid `IllegalMonitorStateException`.
-

Read-Write Locks

When a data structure is **read-heavy** (e.g., stock quotes, configuration caches, product catalogs), blocking every reader while **one** writer updates a single value wastes parallelism.

`ReadWriteLock` solves this by giving us **two** independent locks:

- **Read lock** – many threads can hold it **simultaneously** as long as no thread is writing.
- **Write lock** – exclusive; once held, it blocks **all** other readers and writers.

Real-life Analogy

Think of a **public library**:

- Visitors may **read** books at the same time without disturbing one another.
- When the librarian must **update** (re-shelve) a book, the aisle is temporarily closed—nobody may read or shelve until the update ends.

Code – Live Stock Price Feed

```
import java.util.concurrent.locks.*;
```

```
// Shared stock price data guarded by a ReadWriteLock
```

```

class StockData {

    private double price = 100.0; // initial price

    private final ReadWriteLock lock = new ReentrantReadWriteLock();

    // Writer: updates price

    public void updatePrice(double newPrice) {

        lock.writeLock().lock(); // acquire write lock

        try {

            System.out.printf("%s updating price to %.2f%n",
                               Thread.currentThread().getName(), newPrice);

            price = newPrice;

        } finally {

            lock.writeLock().unlock(); // release write lock

        }

    }

    // Reader: reads price

    public void readPrice() {

        lock.readLock().lock(); // acquire read lock

        try {

            System.out.printf("%s read price: %.2f%n",

```

```
        Thread.currentThread().getName(), price);  
  
    } finally {  
  
        lock.readLock().unlock(); // release read lock  
  
    }  
  
}  
  
}
```

Key Takeaways

- **Separation of Read and Write Access:** The class uses a **ReadWriteLock** to separate read operations (**readPrice()**) from write operations (**updatePrice()**), allowing for better concurrency.
 - **Multiple Readers Allowed Simultaneously:** Multiple threads can safely call **readPrice()** at the same time without blocking each other, as long as no thread is writing.
 - **Only One Writer Allowed at a Time:** When **updatePrice()** is called, it uses **writeLock()**, which blocks all other readers and writers until the update is complete.
 - **Thread Safety Guaranteed:** Locks ensure that reads and writes to the shared **price** variable happen without race conditions, preserving data consistency.
 - **Use Case Fit:** This pattern is ideal for **read-heavy** systems like stock market apps, product catalogs, or analytics dashboards, where frequent reads and infrequent updates are expected.
 - **Clean and Safe Resource Handling:** Both methods use **try-finally** blocks to **guarantee** that the lock is always released, even if an exception occurs during the operation.
-

Semaphores

The Problem

Imagine a scenario where a premium platform like **TUF+** allows access to only a limited number of devices per account. However, users may try to **share credentials** and log in from multiple devices simultaneously, violating this policy. This leads to uncontrolled concurrency and degraded service quality for genuine users.

Real World Analogy

To understand how we can control such access, think of a **token bucket system** at the entrance of a private club. The club has exactly N tokens, and anyone who wishes to enter must pick one up. Once inside, they use the club facilities, and upon leaving, **return the token** to the bucket. If there are no tokens left, new visitors must wait until someone exits. This simple system ensures that **no more than N people are inside at any time**.

The Solution

The solution to the above discussed problem is a **Semaphore** in Java which is a concurrency tool that maintains a fixed number of permits—like tokens. A thread must **acquire** a permit to proceed and **release** it after completing its task. If no permits are available, it either waits or fails based on the method used. This helps **limit the number of threads** accessing a resource at once, making it ideal for enforcing device limits, rate limiting, or managing connection pools.

Code

```
import java.util.concurrent.Semaphore;

// Enforces a max-devices policy for a TUF+ account

class TUFPlusAccount {
```

```
// fixed number of login permits (tokens)

private final Semaphore deviceSlots;


// constructor sets permit count

public TUFPlusAccount(int maxDevices) {

    this.deviceSlots = new Semaphore(maxDevices);

}


// user attempts to log in

public boolean login(String user) throws InterruptedException {

    System.out.println(user + " trying to log in...");


    // try to grab a permit immediately (non-blocking)

    if (deviceSlots.tryAcquire()) {

        System.out.println(user + " successfully logged in.");

        return true;

    } else {

        System.out.println(user + " denied login - too many devices.");

        return false;

    }

}
```

```
}
```

```
// user logs out and returns the permit
```

```
public void logout(String user) {
```

```
    System.out.println(user + " logging out.");
```

```
    deviceSlots.release(); // release permit for next device
```

```
}
```

```
}
```

```
// Quick demo – 2 device limit
```

```
class Main {
```

```
    public static void main(String[] args) throws InterruptedException {
```

```
        TUFPlusAccount account = new TUFPlusAccount(2);
```

```
        Thread u1 = new Thread(() -> trySession(account, "User-A"));
```

```
        Thread u2 = new Thread(() -> trySession(account, "User-B"));
```

```
        Thread u3 = new Thread(() -> trySession(account, "User-C")); //
```

```
should fail first
```

```
        u1.start(); u2.start();
```

```
        Thread.sleep(100); // ensure first two log in
```

```
        u3.start();
```

```

        u1.join(); u2.join(); u3.join();
    }

    // helper for a user session
    private static void trySession(TUFPlusAccount acc, String name) {
        try {
            if (acc.login(name)) {
                Thread.sleep(500); // simulate watching a video
                acc.logout(name);
            }
        } catch (InterruptedException ignored) { }
    }
}

```

Key Semaphore Methods

- **tryAcquire()**: Attempts to grab a permit immediately; returns true or false (non-blocking — does not wait if no permit is available).
- **acquire()**: Blocks until a permit is available.
- **release()**: Returns a permit, incrementing the semaphore's count.

Key Takeaways

- **Fixed-size gate**: only N **simultaneous** log-ins are allowed; extras are rejected or block.

- **Fairness Option:** `new Semaphore(n, true)` can grant permits FIFO if you need fairness.
- **Non-blocking vs blocking:** choose `tryAcquire()` for fail-fast behaviour, or `acquire()` when the thread should wait.
- **Always release:** failing to call `release()` will leak permits and eventually lock everyone out.

Use Cases

- Database Connection Pools
- API Rate Limiting
- Parking Lot Capacity
- Concurrent Task Execution

These scenarios require **precise concurrency limits**, which semaphores handle better than basic locks.

Comparing Monitors, Reentrant Locks, and Semaphores

To wrap up this article, here's a quick summary comparing **Monitors**, **Reentrant Locks**, and **Semaphores** — three core concurrency tools in Java. Each has different strengths, use cases, and trade-offs. The comparison below is based on key characteristics and practical applications:

Feature	Monitor (<code>synchronized</code>)	ReentrantLock	Semaphore
---------	--	---------------	-----------

Type	Intrinsic	Explicit Lock API	Permit-based access
Concurrency Limit	1 thread	1 thread	N threads
Reentrant	✓	✓	✗
Timeout Support	✗	✓	✓
Wait/Notify	✓ (<code>wait/notify</code>)	✓ (<code>Condition.await()</code>)	✗
Best Use Case	Simple mutual exclusion	Fine-grained locking	Limiting concurrent access
Ownership Model	Thread-bound	Thread-bound	No ownership (permits only)
Example Use Case	Producer-Consumer	BMS system	DB connection pool

This comparison helps clarify when to use what:

- Use **Monitors** for simple thread coordination and mutual exclusion.
- Use **ReentrantLocks** when you need **flexibility, timeouts, or multiple condition variables**.
- Use **Semaphores** when you want to **control the number of concurrent accesses**, not just mutual exclusion.

Each primitive is powerful when used in the right scenario — understanding their differences ensures safer and more scalable concurrency design.

Conclusion

As we conclude, it's important to understand that all these tools — whether it's a simple monitor, a reentrant lock, or a semaphore — serve one underlying purpose: **safe coordination in a concurrent world**. These aren't just utilities to pick and choose from casually.

In reality, **synchronization is not optional — it's infrastructure**. It's what keeps threads from colliding, ensures data stays consistent, and allows systems to scale without breaking. Much like electricity powers everything silently in the background, synchronization quietly powers every reliable multithreaded application.